ProjectMind

An Evidence-Frontier and Authority-Governed Project Continuity Architecture for AI Coding Agents

Agim Haxhijaha

July 13, 2026

Contents

Publication Record

Author: Agim Haxhijaha, Independent Researcher **Edition:** v4.0 Public Research Edition **Publication series:** Independent Research Publication No. 4 **Publication date:** July 13, 2026 **ORCID:** To be inserted after author verification **DOI:** To be assigned or reserved before public release **Status:** Proposed architecture; not implemented, validated, patented, peer reviewed, certified, or production-ready.

Recommended Citation

Haxhijaha, Agim. ProjectMind: An Evidence-Frontier and Authority-Governed Project Continuity Architecture for AI Coding Agents. v4.0 Public Research Edition. Independent Research Publication No. 4, 2026. DOI to be inserted after reservation.

AI-Assistance Disclosure

Generative AI tools assisted with research synthesis, prior-art comparison, consolidation, language editing, consistency checking, and publication preparation. The named author directed the work and is responsible for the publication decision, claims, omissions, and released content.

Abstract

AI coding agents can retrieve source code while still operating with stale, branch-incomparable, weakly supported, or unauthorized project knowledge. ProjectMind is a proposed local-first architecture that treats agent context as a versioned, verifiable artifact. For a target Git commit it computes a canonical Evidence Frontier, maintains claims with separate authenticity, confidence, authority, freshness, and completeness dimensions, records authority changes in an append-only ledger, and compiles task-specific knowledge into an immutable Change Consequence Contract. After a candidate is frozen, ProjectMind independently reconstructs the observed graph and test delta, reconciles that delta with the prediction, and emits a Continuity Receipt binding the repository state, supplied context, policy state, evidence, and outcome. The proposal builds on established code graphs, temporal agent memory, test-impact analysis, MCP, provenance, policy, and attestation systems. Its research contribution is the linked commit-DAG continuity-control chain, not the novelty of those components individually. No implementation or empirical result is claimed.

Keywords

AI coding agents; project continuity; repository knowledge graph; evidence frontier; authority governance; change consequence contract; context receipt; continuity receipt; test impact analysis; MCP; software provenance; human oversight.

Reading and Claim Rule

Later formal-contribution sections refine earlier general architecture language. When a general statement conflicts with the Evidence Frontier, Authority Promotion Ledger, CCC, delta-reconciliation, or receipt rules, the narrower formal rule controls.

Section Guide

Section 0 - Executive Definition

ProjectMind is a proposed local-first project-continuity and evidence-governance architecture for AI-assisted software development.

It maintains a commit-DAG-aware model of a repository, derives an explicit **Evidence Frontier** for a chosen commit, distinguishes information quality from policy authority, records every authority change in an append-only **Authority Promotion Ledger**, and compiles task-specific knowledge into an immutable **Change Consequence Contract**. After a candidate change is frozen, ProjectMind compares the predicted graph delta with the observed graph delta and emits a verifiable **Continuity Receipt**.

The architecture addresses a narrow failure: an agent may retrieve correct code while still acting on stale, branch-incomparable, weakly supported, or unauthorized project knowledge. ProjectMind therefore treats context as a versioned technical artifact, not as an informal prompt attachment.

The proposed flow is:

```
repository commit DAG
  -> admissible evidence
  -> Evidence Frontier digest
  -> authority-governed active claims
  -> bounded context plus Context Receipt
  -> Change Consequence Contract
  -> candidate change
  -> observed graph and test delta
  -> predicted/observed reconciliation
  -> Continuity Receipt
  -> independent acceptance system or human review
```

ProjectMind does not claim consciousness, complete project understanding, complete static analysis, guaranteed correctness, patentability, legal compliance, implementation, validation, peer review, or production readiness.

Section 1 - Product Identity

1.1 Product Name

ProjectMind

1.2 Public Title

ProjectMind: An Evidence-Frontier and Authority-Governed Project Continuity Architecture for AI Coding Agents

1.3 Tagline

Verifiable Project Context Before and After AI-Assisted Change

1.4 Category

- AI coding governance;
- repository intelligence;
- project continuity and memory;
- software-change consequence analysis;
- provenance-aware context infrastructure;
- test and invariant governance.

1.5 Short Description

ProjectMind creates a version-bound, evidence-backed project state for an AI coding task, records exactly what context was supplied, predicts the expected structural consequences, and verifies those consequences after the change.

1.6 Public Claim Boundary

ProjectMind is a proposed architecture. The name does not imply machine consciousness. No claim of firstness, patentability, implementation, benchmarked performance, certification, or production readiness is made.

Section 2 - Problem Statement

Coding agents often complete local tasks while damaging the wider project.

The root problem is not merely weak code generation. It is missing project continuity.

A human engineer working on step 50 remembers:

- what step 1 established;
- why step 12 chose a specific API;
- which shortcut from step 27 is temporary;
- which failure happened at step 39;
- which module planned for step 100 depends on an abstraction created today;
- what the product is intended to become;
- which tradeoffs are already accepted;
- what “success” means beyond compiling.

A coding agent commonly sees:

- a current prompt;
- a limited set of files;
- a compressed or stale conversation;
- partial repository search results;
- local test output;
- no durable consequence model.

This produces recurring failure classes:

1. **Local optimization failure** - the current feature works, but shared architecture degrades.
2. **Historical amnesia** - the agent reverses a previous decision because the reason is missing.
3. **Future blindness** - the agent closes an extension point needed by a later roadmap item.
4. **Invariant blindness** - the agent does not know which tests encode critical behavior.
5. **Dependency blindness** - the agent modifies a hub module without recognizing downstream impact.
6. **Reward blindness** - passing the current test is treated as success even when project health declines.

7. **Failure repetition** - a later session repeats a patch pattern that previously caused regressions.
8. **Context poisoning** - repository text, generated files, or tool responses inject misleading instructions.
9. **False certainty** - model-generated summaries become accepted as facts without evidence.

ProjectMind exists to convert fragmented project evidence into durable, queryable, authority-graded continuity.

Section 3 - Core Product and Research Thesis

A coding agent should receive neither a raw repository dump nor a memory summary whose branch, freshness, provenance, authority, and omissions are unknown. It should receive a reproducible project-state view bound to the exact repository state on which the task operates.

ProjectMind proposes five linked mechanisms:

1. **Evidence Frontier** - a deterministic manifest of admissible evidence for a target commit in a non-linear Git history.
2. **Authority Promotion Ledger** - an append-only record of every transition from proposed knowledge to evidenced or enforceable knowledge.
3. **Change Consequence Contract** - an immutable, task-specific declaration of relevant invariants, expected impact, required evidence, unknowns, and review triggers.
4. **Delta Reconciliation** - deterministic comparison of predicted and observed graph, dependency, test, and policy effects.
5. **Context and Continuity Receipts** - hash-bound evidence of what the agent was told and what the final candidate actually changed.

The research hypothesis is that this chain reduces silent context drift and makes project-memory governance independently auditable. That hypothesis remains to be tested; this document supplies falsifiable requirements rather than results.

Section 4 - Prior Art, Research Position, and Hardening Decisions

The public edition narrows its contribution after comparison with current work. Persistent code graphs, repository knowledge graphs, temporal agent memory, impact analysis, test selection, event logs, MCP context servers, policy engines, and software attestations all have substantial prior art.

Existing work	Established capability
Codebase-Memory	Persistent Tree-sitter knowledge graph, MCP, impact queries
Repository Intelligence Graph	Deterministic evidence-backed build and test graph
PROJECTMEM	Local event-sourced memory, MCP summaries, pre-action warnings
KGCompass and Prometheus	Repository knowledge graphs for repair and issue resolution
Zep/Graphiti	Temporal knowledge-graph agent memory
CodePlan	Incremental dependency and may-impact planning
Regression-test-selection research	Change-to-test selection and assertion-level selection
W3C PROV, in-toto, SLSA, DSSE	Provenance and signed attestations

ProjectMind therefore positions its contribution as the **integrated, commit-DAG-bound continuity-control chain**, especially the deterministic Evidence Frontier, authority-transition

ledger, immutable consequence contract, predicted/observed delta reconciliation, and linked receipts.

Hardening decisions in v4.0:

- Git-native mode is the primary MVP; verified external change events are an optional authority upgrade.
- Source authenticity, evidence confidence, policy authority, freshness, and completeness are separate dimensions.
- A valid signature proves origin and integrity, not factual correctness.
- Claims are modeled separately from entities and may cite multiple evidence objects.
- Validity follows Git ancestry and reachability, not a linear commit interval.
- Missing or weaker evidence cannot silently reduce review requirements.
- Context truncation and omitted source classes are part of the receipt.
- SQLite must use a version containing the 2026 WAL-reset correction.
- The earlier seven-day plan is reclassified as a feasibility spike, not a full implementation schedule.

Section 5 - Non-Negotiable Design Principles

1. Repository state is identified by object IDs and commit reachability.
2. Context is a reproducible artifact, not an informal summary.
3. Every active claim cites one or more evidence objects.
4. Authenticity, confidence, authority, freshness, and completeness remain separate.
5. Cryptographic verification never proves semantic truth by itself.
6. Model output begins as PROPOSED and cannot promote itself.
7. Blocking authority requires explicit policy or recorded human approval.
8. Authority transitions are append-only and explainable.
9. Branch-incomparable evidence is not silently combined.
10. Stale, conflicted, or incomplete evidence is visible.
11. Reduced evidence cannot produce a less cautious decision.
12. Expected consequences are frozen before candidate evaluation.
13. Observed consequences are independently reconstructed after the change.
14. Predicted/observed mismatch is never hidden inside a score.
15. Every context response declares limits and omissions.
16. Repository text, tool descriptions, and generated summaries are untrusted content unless separately authorized.
17. The MVP MCP surface is local, read-only, bounded, and non-sampling.
18. ProjectMind does not modify or merge product code.
19. An external acceptance system or a human retains change authority.
20. The graph must be reproducible from retained evidence and pinned extractors.
21. Every release publishes limitations and proof receipts.
22. No performance, security, or novelty claim is made without evidence.

Section 6 - System Boundary

6.1 ProjectMind Controls

- evidence ingestion and quarantine;
- Git commit-DAG modeling;
- active Evidence Frontier computation;
- temporal claims and graph views;
- authority-transition recording;
- structural and test-impact analysis;
- bounded context delivery and Context Receipts;
- Change Consequence Contract compilation;

- predicted/observed delta reconciliation;
- Continuity Receipt generation;
- policy-context export.

6.2 ProjectMind Does Not Control

- the coding model;
- agent file-system or kernel isolation;
- final repository acceptance or merge;
- semantic correctness of business logic;
- complete language, runtime, or build understanding;
- the truth of human-provided assertions;
- legal compliance or patentability.

6.3 Independent Acceptance Boundary

ProjectMind provides contracts, evidence, and receipts. A human reviewer, CI policy, repository rule, IntentLock adapter, or another independent system decides whether the candidate may be accepted.

Section 7 - Reference Architecture

ProjectMind v4.0 contains ten logical planes:

1. **Evidence Plane** - immutable source artifacts and normalized metadata.
2. **Ingestion Plane** - validation, canonicalization, deduplication, and quarantine.
3. **Commit-DAG Plane** - repository identity, commits, parents, reachability, and branch comparison.
4. **Claim Graph Plane** - entities, relations, claims, evidence links, and materialized active views.
5. **Frontier Plane** - admissibility, invalidation, conflict, completeness, and Evidence Frontier digests.
6. **Authority Plane** - promotion policy and append-only Authority Promotion Ledger.
7. **Context Plane** - adaptive retrieval, briefs, MCP, and Context Receipts.
8. **Consequence Plane** - impact sets, invariants, tests, unknowns, and immutable Change Consequence Contracts.
9. **Reconciliation Plane** - observed extraction, delta comparison, and Continuity Receipts.
10. **Governance Plane** - approvals, keys, retention, privacy, audit, and release evidence.

Section 8 - Evidence Quality and Authority Vector

ProjectMind replaces a single trust ladder with an orthogonal vector:

```
Q = (authenticity, confidence, authority, freshness, completeness)
```

8.1 Authenticity

- UNVERIFIED
- HASH_BOUND
- SIGNATURE_VERIFIED
- IDENTITY_ATTESTED

Authenticity answers who or what produced the artifact and whether bytes changed. It does not prove that the artifact is correct.

8.2 Evidence Confidence

- EXACT
- HIGH
- MEDIUM
- LOW
- UNKNOWN
- CONFLICTED

8.3 Policy Authority

- NONE
- WARN
- REVIEW
- BLOCK
- HARD_STOP

8.4 Freshness

- CURRENT
- POSSIBLY_STALE
- STALE
- INVALIDATED
- HISTORICAL

8.5 Completeness

- COMPLETE_FOR_DECLARED_SCOPE
- PARTIAL
- MISSING_REQUIRED_CLASS
- UNKNOWN

8.6 Safety Rule

No transformation may infer higher authority merely from higher authenticity or confidence. A downgrade in freshness or completeness cannot reduce an existing review requirement unless a separately authorized policy explicitly resolves the condition and records the reason.

Section 9 - Change Telemetry and Evidence Modes

9.1 GIT_NATIVE Mode

The MVP starts from Git commits and diffs, code indexes, build metadata, tests, coverage, and recorded human decisions. Unknown human intent remains UNKNOWN.

9.2 VERIFIED_EVENT Mode

Signed events from IntentLock or another authorized change-control system may upgrade authenticity and attach approved task intent. The schema remains the same.

9.3 Required Event Classes

- task or intent approved;
- candidate frozen;
- test evidence produced;
- candidate accepted or rejected;

- merge completed or reverted;
- human authority transition recorded.

9.4 Delivery

MVP delivery uses atomic file drop and explicit CLI ingestion. Delivery is at-least-once; identity is `source + event_id + canonical_payload_hash`.

Section 10 - Event Envelope and Signature Model

10.1 CloudEvents-Compatible Envelope

```
{
  "specversion": "1.0",
  "id": "IL-EVT-2026-000001",
  "source": "urn:intentlock:repo:sha256:REPO_HASH",
  "type": "dev.intentlock.merge.verified.v1",
  "subject": "MERGE-001",
  "time": "2026-07-10T12:00:00+02:00",
  "datacontenttype": "application/json",
  "dataschema": "urn:projectmind:schema:intentlock-merge-verified:v1",
  "projectmindprojectid": "PM-001",
  "intentlocksessionid": "IL-SESSION-2026-0001",
  "data": {
    "project": {
      "repo_root_hash": "sha256:...",
      "base_commit": "abc123",
      "result_commit": "def456"
    },
    "contract": {
      "contract_id": "IL-2026-0001",
      "contract_hash": "sha256:...",
      "task_hash": "sha256:...",
      "task_summary": "Add Google OAuth login"
    },
    "operations": [
      {
        "operation_id": "OP-001",
        "target_paths": ["src/auth/google.ts"],
        "decision": "ALLOW",
        "envelope_conformant": true,
        "privacy_decision": "CLEAR"
      }
    ],
    "changed_files": [
      {
        "path": "src/auth/google.ts",
        "change_type": "modified",
        "before_hash": "sha256:...",
        "after_hash": "sha256:..."
      }
    ],
    "tests": {
      "overall": "PASS",
      "result_artifacts": [
```

```
        {
          "format": "junit",
          "path": ".intentlock/evidence/junit.xml",
          "hash": "sha256:..."
        }
      ],
      "coverage_artifacts": [
        {
          "format": "lcov",
          "path": ".intentlock/evidence/lcov.info",
          "hash": "sha256:..."
        }
      ]
    },
    "merge": {
      "merge_id": "MERGE-001",
      "merged_diff_hash": "sha256:...",
      "inverse_patch_hash": "sha256:..."
    },
    "attestation": {
      "status": "VALID",
      "statement_hash": "sha256:..."
    }
  }
}
```


10.2 DSSE Envelope

When signing is enabled, the CloudEvents JSON bytes are carried as the DSSE payload.

Verification:

1. load trusted key policy;
2. verify DSSE signature before parsing trusted fields;
3. verify payload type;
4. parse CloudEvents;
5. validate schema;
6. verify repository and commit binding;
7. compute data hash;
8. ingest or quarantine.

10.3 Signature Modes

- REQUIRED
- OPTIONAL_LOCAL
- DISABLED_DEVELOPMENT

10.4 Key Handling

ProjectMind stores public verification keys, key IDs, trust policy, and rotation history. It does not require IntentLock private keys.

Section 11 - Evidence Ingestion and Idempotency

Pipeline:

1. receive artifact;
2. enforce maximum size;
3. detect envelope type;
4. verify signature;
5. validate schema;
6. verify repository binding;
7. verify commit;
8. calculate digest;
9. check idempotency;
10. store raw event;
11. start database transaction;
12. create or update graph facts;
13. commit;
14. schedule targeted extraction;
15. record receipt;
16. move event to processed or quarantine.

A graph update is atomic. No partial event becomes visible.

Quarantine reasons include invalid signature, untrusted key, schema mismatch, event collision, missing commit, repository mismatch, malformed path, path escape, hash mismatch, oversized payload, and unsupported encoding.

Section 12 - Project State Claim Graph

ProjectMind separates stable entities from claims about those entities.

12.1 Entity Types

PROJECT, REPOSITORY, COMMIT, BRANCH, RELEASE, GOAL, ROADMAP_ITEM, TASK, CONTRACT, APPROVAL, CHANGE, MODULE, PACKAGE, FILE, SYMBOL, ENDPOINT, DATABASE_ENTITY, CONFIGURATION, BUILD_TARGET, TEST_SUITE, TEST_CASE, ASSERTION, INVARIANT, DECISION, RISK, FAILURE, EVIDENCE, TOOL, EXTRACTOR, UNKNOWN.

12.2 Claim Form

A claim is:

```
(subject, predicate, object-or-value, scope, quality-vector, lifecycle)
```

Examples include `FILE IMPORTS PACKAGE`, `TEST_CASE EXECUTES SYMBOL`, and `INVARIANT PROTECTS MODULE`.

12.3 Evidence Is Many-to-Many

A claim may cite several artifacts, and one artifact may support or contradict several claims. Evidence links therefore use a join relation with SUPPORTS, CONTRADICTS, INVALIDATES, or SUPERSEDES semantics.

12.4 Materialized Graph View

Dependency and impact queries operate on a materialized view of claims admissible under the selected Evidence Frontier. The raw claim ledger is retained even when a claim is rejected, stale, superseded, or branch-incomparable.

Section 13 - Commit-DAG Time, Scope, and Evidence Frontiers

Git history is a directed acyclic graph, not a single timeline. ProjectMind uses ancestry and reachability instead of assuming that `valid_from` and `valid_to` identify one global interval.

13.1 Claim Scope

Each claim records:

- repository identity;
- introducing commit;
- optional invalidating commit;
- branch or ref observation, if relevant;
- system observation time;
- evidence environment digest;
- extractor and schema versions.

A claim is eligible at target commit `C` only when its introducing commit is reachable from `C` and no reachable invalidation removes it. Evidence from a branch incomparable with `C` is excluded or explicitly marked comparative.

13.2 Evidence Frontier

The Evidence Frontier for `(repository, target_commit, policy_version)` is the canonical manifest of every admissible evidence object and active claim after ancestry, invalidation, conflict, freshness, and required-source checks.

The manifest is serialized with RFC 8785 JSON canonicalization and hashed. Every brief, MCP response, consequence contract, and receipt carries this digest.

13.3 Bitemporal Semantics

ProjectMind records repository-valid scope and system-observation time. It can answer both ”what claims apply to commit C?” and ”what did ProjectMind know at time T about commit C?”

Section 14 - Embedded Store and Integrity Model

Canonical local store: `.projectmind/graph.db`.

Use SQLite STRICT tables, foreign keys, one application writer, bounded reader transactions, WAL checkpoints, backups, migration checksums, and startup integrity verification.

Because SQLite documented and fixed a rare WAL-reset corruption bug in 2026, the release must use SQLite 3.51.3 or later, or an official corrected backport such as 3.50.7 or 3.44.6. Dependency pinning must verify the embedded SQLite version.

Core relational model:

```
CREATE TABLE evidence (
  evidence_id TEXT PRIMARY KEY,
  kind TEXT NOT NULL,
```

```
  canonical_hash TEXT NOT NULL,
  authenticity TEXT NOT NULL,
  observed_at TEXT NOT NULL,
  metadata_json TEXT NOT NULL
) STRICT;

CREATE TABLE commits (
  repository_id TEXT NOT NULL,
  commit_id TEXT NOT NULL,
  observed_at TEXT NOT NULL,
  PRIMARY KEY (repository_id, commit_id)
) STRICT;

CREATE TABLE commit_parents (
  repository_id TEXT NOT NULL,
  commit_id TEXT NOT NULL,
  parent_id TEXT NOT NULL,
  parent_position INTEGER NOT NULL,
  PRIMARY KEY (repository_id, commit_id, parent_position)
) STRICT;

CREATE TABLE claims (
  claim_id TEXT PRIMARY KEY,
  subject_id TEXT NOT NULL,
  predicate TEXT NOT NULL,
  object_json TEXT NOT NULL,
  introduced_commit TEXT,
  invalidated_commit TEXT,
  confidence TEXT NOT NULL,
  authority TEXT NOT NULL,
  freshness TEXT NOT NULL,
  completeness TEXT NOT NULL,
  lifecycle TEXT NOT NULL,
  schema_version INTEGER NOT NULL
) STRICT;

CREATE TABLE claim_evidence (
  claim_id TEXT NOT NULL,
  evidence_id TEXT NOT NULL,
  relation TEXT NOT NULL,
  PRIMARY KEY (claim_id, evidence_id, relation),
  FOREIGN KEY (claim_id) REFERENCES claims(claim_id),
  FOREIGN KEY (evidence_id) REFERENCES evidence(evidence_id)
) STRICT;

CREATE TABLE authority_transitions (
  transition_id TEXT PRIMARY KEY,
  claim_id TEXT NOT NULL,
  prior_state_hash TEXT NOT NULL,
  new_state_hash TEXT NOT NULL,
  actor_or_policy TEXT NOT NULL,
  reason TEXT NOT NULL,
  evidence_frontier_hash TEXT NOT NULL,
  occurred_at TEXT NOT NULL,
  signature_ref TEXT
) STRICT;
```

Agents cannot submit arbitrary SQL. Reachability caches are derived data and must be invalidated when new commits or refs are observed.

Section 15 - Code-Intelligence Extractor Fabric

No single parser captures all truth.

Every extractor declares ID, version, languages, build systems, node and edge types, inputs, limits, license, executable path, resource limits, timeout, output schema, and security classification.

Precedence:

1. compiler-accurate index;
2. SCIP;
3. language-server or build metadata;
4. Tree-sitter;
5. heuristic inference;
6. LLM interpretation.

External extractors run read-only, network-denied by default, with CPU, memory, and timeout limits.

Section 16 - SCIP Adapter

SCIP provides standardized definitions, references, implementations, and symbol relationships.

SCIP is a preferred semantic input, not the sole parser.

Validation:

- protobuf parses;
- project root matches;
- commit matches where available;
- paths stay inside repository;
- symbol IDs normalize;
- indexer version records;
- unsupported fields remain as raw evidence.

If no indexer is available, use Tree-sitter and build adapters, mark semantic coverage PARTIAL, and disclose reduced confidence.

Recommended first semantic path: TypeScript/JavaScript through `scip-typescript`.

Section 17 - Tree-sitter Adapter

Tree-sitter provides resilient incremental syntax extraction.

It produces syntax trees, definitions, imports, exports, declarations, coarse call references, test structure, and changed-range data. It does not automatically provide compiler-accurate type resolution.

Recommended built-in MVP grammars:

- TypeScript
- JavaScript
- TSX
- JSX

- JSON
- optional Rust

Each grammar requires pinned source, license, checksum, fixture, supported version range, update procedure, and SBOM entry.

Section 18 - Build-System and Manifest Adapters

Build systems reveal packages, targets, generated code, tests, compilation units, runtime entry points, optional features, and deployment topology.

MVP adapters:

- `package.json`
- npm, pnpm, and Yarn lockfiles
- `tsconfig.json`
- Git metadata
- generic JUnit XML
- LCOV
- Istanbul JSON summary where available

Next adapters:

- Cargo metadata
- Python metadata
- CMake File API and CTest
- Go modules and `go list`
- Gradle
- Maven
- Bazel
- .NET project files

Dependency classes:

LOCAL_SOURCE, WORKSPACE_PACKAGE, DIRECT_EXTERNAL, TRANSITIVE_EXTERNAL, BUILD_ONLY, TEST_ONLY, OPTIONAL, GENERATED, UNKNOWN.

Section 19 - Dynamic Test-Evidence Engine

19.1 Purpose

Determine which tests exercise which code and whether relevant tests passed.

19.2 Evidence Inputs

- JUnit XML
- test-runner JSON
- LCOV
- Istanbul coverage JSON
- coverage.py JSON with contexts
- CTest metadata
- future language adapters

19.3 Relationship Types

- TEST_DECLARES_TARGET
- TEST_IMPORTS

- TEST_EXECUTES_FILE
- TEST_EXECUTES_RANGE
- TEST_COVERS_SYMBOL
- TEST_ASSERTS_BEHAVIOR
- TEST_FAILED_ON_CHANGE
- TEST_PASSED_AT_COMMIT

19.4 Coverage Semantics

Aggregate coverage supports suite-to-file and suite-to-line evidence.

Per-test coverage contexts support test-case-to-file and test-case-to-line evidence.

ProjectMind must not invent per-test relationships from aggregate coverage.

19.5 Test Result Freshness

A test result is current only for the recorded commit, environment, test command, and dependency state. A later code change may invalidate it.

19.6 Negative Evidence

ProjectMind records skipped tests, quarantined tests, flaky tests, unexecuted tests, coverage failures, test deletion, assertion weakening, and snapshot updates.

A passing suite with skipped relevant tests is not equivalent to full preservation.

Section 20 - Test-Driven Invariant Governance

Tests provide several different kinds of evidence and must not be collapsed into one meaning.

20.1 Evidence Levels

- A test name may create a LOW-confidence PROPOSED claim.
- A parsed assertion may create an EVIDENCED claim about the exact checked value.
- Aggregate coverage may create suite-to-file or suite-to-line execution claims.
- Per-test contexts may create test-case-to-range execution claims.
- A passing result is valid only for the bound commit, command, dependencies, and environment digest.
- A human or authorized deterministic policy is required for BLOCK authority.

20.2 Invariant Claim Example

```
{
  "claim_id": "CLAIM:INV-PAY-001:v3",
  "subject": "INVARIANT:INV-PAY-001",
  "predicate": "PROTECTS",
  "object": {"modules": ["payments", "auth"]},
  "introduced_commit": "def456",
  "quality": {
    "authenticity": "IDENTITY_ATTESTED",
    "confidence": "HIGH",
    "authority": "BLOCK",
    "freshness": "CURRENT",
    "completeness": "COMPLETE_FOR_DECLARED_SCOPE"
```

```
  },
  "lifecycle": "ENFORCED",
  "required_tests": ["payments.auth.rejects-unauthorized"],
  "evidence_ids": ["E:junit:...", "E:assertion:...", "E:approval:..."]
}
```

20.3 Negative Evidence

Deleted, skipped, quarantined, flaky, weakened, or non-executed tests reduce freshness or completeness. Snapshot updates and assertion weakening are recorded explicitly. A passing suite with missing required tests cannot satisfy the same contract as a complete run.

Section 21 - Architecture-Decision Extraction

21.1 Purpose

Create reviewable ADR candidates from approved engineering work.

21.2 Trigger Conditions

- new package or module;
- public API change;
- schema change;
- dependency addition;
- auth or permission change;
- storage model change;
- infrastructure change;
- cross-module dependency;
- high blast radius;
- explicit human approval;
- repeated pattern across merges.

21.3 Deterministic ADR Skeleton

Before any LLM is used, ProjectMind creates:

- source event;
- task;
- changed files;
- symbols added or removed;
- dependency delta;
- tests added;
- approvals;
- known consequences;
- unresolved questions.

21.4 Optional LLM Summary

The model may propose title, context, decision statement, apparent rationale, implied alternatives, future consequences, and risks.

21.5 LLM Safety Boundary

- local model preferred;
- cloud disabled by default;

- secrets and sensitive paths excluded;
- input budget capped;
- untrusted text labeled;
- repository instructions never become system instructions;
- output schema validated;
- all claims linked to evidence;
- no auto-promotion;
- no blocking.

21.6 ADR States

- DRAFT_DETERMINISTIC
- PROPOSED_LLM
- ACCEPTED_HUMAN
- REJECTED
- SUPERSEDED
- STALE

Section 22 - Claim Lifecycle and Authority Promotion

22.1 Lifecycle

PROPOSED, EVIDENCED, ENFORCED, REJECTED, CONFLICTED, STALE, SUPERSEDED, QUARANTINED, and UNKNOWN.

22.2 Promotion Preconditions

A transition to ENFORCED requires:

1. a policy rule or identified human authority;
2. the target claim and exact prior-state hash;
3. an Evidence Frontier digest;
4. supporting and contradicting evidence disclosure;
5. an explicit reason and effective scope;
6. an append-only Authority Promotion Ledger entry.

LLM text, test names, comments, and repository instructions cannot promote themselves. Passing tests may satisfy an authorized promotion policy but do not create that policy.

22.3 Conflict and Downgrade

Contradictory active evidence produces CONFLICTED status. Required stale evidence removes or suspends blocking export according to policy and cannot silently pass.

22.4 Monotonic Caution

For the same task and policy, losing evidence, freshness, or completeness cannot change a decision from REVIEW/BLOCK to PASS.

Section 23 - Context Broker, Retrieval Router, and Context Receipts

ProjectMind routes exact symbol questions to semantic indexes, dependency questions to bounded graph traversal, historical questions to commit-DAG claims, governance questions to

the Authority Promotion Ledger, and semantic questions to optional grounded retrieval.

Simple exact queries do not invoke an LLM. Graph expansion is bounded by snapshot, depth, node count, edge types, module boundary, time, and response bytes.

Every response includes:

- target commit and Evidence Frontier digest;
- query class and canonical query hash;
- returned claim IDs and evidence IDs;
- authority and quality vectors;
- traversal and ranking configuration;
- omitted source classes and truncation limits;
- unknowns and conflicts;
- MCP tool-manifest hash;
- response digest.

These fields form a **Context Receipt**. The receipt proves which project-state view was supplied; it does not prove that the agent read or obeyed it.

Section 24 - MCP Interface and Security Profile

MCP standardizes context and tool integration but does not itself enforce the ProjectMind security model.

MVP profile:

- direct local stdio transport;
- read-only resources and tools;
- no server sampling, elicitation, shell, network, file writes, arbitrary SQL, or dynamic tool installation;
- explicit repository-root binding;
- path canonicalization and symlink-escape checks;
- fixed response budgets, timeouts, and cancellation;
- tool names, descriptions, schemas, version, binary digest, and configuration included in the manifest hash;
- no hidden instruction authority;
- every call produces a Context Receipt.

A local MCP server is executable code and remains a supply-chain and host-security risk. Distribution therefore requires digest pinning, signed releases, least privilege, and an operating-system sandbox where available.

Approved MVP tools remain structural, historical, explanatory, and read-only.

Section 25 - Continuity Brief Generator

25.1 Purpose

Provide a human-readable starting point even when MCP is unavailable.

25.2 Brief Sections

1. Task
2. Repository snapshot
3. Current project phase
4. Related goals
5. Relevant prior changes
6. Enforced invariants

7. Evidenced constraints
8. Proposed warnings
9. Structural dependencies
10. High-risk hubs
11. Safe change zone
12. Dangerous change zone
13. Required tests
14. Relevant failures
15. Roadmap implications
16. Approval triggers
17. Unknowns
18. Context limits
19. Required post-change checks

25.3 Size Policy

Default brief is concise. Expanded evidence is available through MCP. Raw secrets and full file bodies are excluded by default.

25.4 Example

```
TASK
Add invoice export to the dashboard.

SNAPSHOT
Commit: def456
Graph snapshot: PM-SNAP-0042

ENFORCED
INV-AUTH-002  -  Invoice access requires an authenticated session.
INV-PII-004  -  User email may not be written to application logs.

EVIDENCED
dashboard/invoices depends on payments/invoice-service.
invoice-service imports auth/session.
Three tests exercised the changed payment path at this commit.

PROPOSED
ADR candidate suggests provider-specific billing logic should remain behind an
 ↪  adapter.
This is non-blocking until approved.

DANGEROUS ZONE
src/auth/**
src/payments/provider/**
database migrations

REQUIRED TESTS
payments authorization suite
invoice export suite
privacy logging check

UNKNOWN
No current evidence establishes performance behavior for exports above 10,000 rows.
```

Section 26 - Change Consequence Contract

Before candidate evaluation, ProjectMind compiles an immutable Change Consequence Contract (CCC).

Inputs include the task digest, base commit, Evidence Frontier, authorized claims, proposed scope, dependency closure, impacted tests, protected invariants, known failures, roadmap dependencies, and explicit unknowns.

```
{
  "schema": "dev.projectmind.ccc/v1",
  "contract_id": "PM-CCC-0042",
  "repository": "sha256:...",
  "base_commit": "abc123",
  "task_digest": "sha256:...",
  "evidence_frontier_digest": "sha256:...",
  "policy_version": "policy-7",
  "allowed_scope": ["src/invoices/**", "tests/invoices/**"],
  "predicted_delta": {
    "required_relations": [],
    "allowed_relations": [],
    "forbidden_relations": []
  },
  "required_tests": ["invoice-auth", "invoice-export"],
  "review_triggers": ["PUBLIC_API_REMOVAL", "AUTH_PATH_CHANGE"],
  "unknowns": ["large-export performance envelope"],
  "expires_on_frontier_change": true,
  "contract_digest": "sha256:..."
}
```

The CCC is canonicalized and frozen before the observed candidate delta is analyzed. It may be signed by a human or policy service, but signature validity does not make its predictions correct.

Section 27 - Reward, Failure, and Learning Model

27.1 Purpose

Measure project-level success rather than only task completion.

27.2 Reward Dimensions

- task completion;
- invariant preservation;
- required test success;
- dependency stability;
- architecture consistency;
- roadmap compatibility;
- security preservation;
- privacy preservation;
- rollback readiness;
- documentation alignment;
- change minimality;
- repeated-failure avoidance.

27.3 Reward Contract

Each dimension includes value, calculation, evidence, confidence, and missing data.

27.4 Example

```
{
  "reward_id": "PM-REWARD-0042",
  "snapshot_id": "PM-SNAP-0042",
  "task_id": "TASK-050",
  "scores": {
    "task_completion": 1.0,
    "invariant_preservation": 1.0,
    "required_test_success": 0.8,
    "dependency_stability": 0.7,
    "roadmap_compatibility": 1.0,
    "rollback_readiness": 1.0
  },
  "missing_evidence": [
    "Performance behavior for large exports not measured."
  ],
  "decision": "PASS_WITH_WARNINGS"
}
```

27.5 Failure Memory

A failure record includes task, patch or merge, symptoms, root cause if known, affected nodes, supporting tests, rollback, repeated pattern, avoidance guidance, confidence, and human review.

27.6 No Autonomous Self-Modification

Failure memory may recommend policies. It cannot silently change enforcement.

Section 28 - Integration and Portable Acceptance Handoff

ProjectMind is useful without a companion product.

28.1 Git-Native Flow

1. Bind task to base commit.
2. Compute Evidence Frontier.
3. Deliver bounded context and Context Receipt.
4. Compile Change Consequence Contract.
5. External agent or human creates a candidate.
6. Freeze candidate commit or tree.
7. Re-index affected scope and ingest tests.
8. Reconcile predicted and observed deltas.
9. Emit Continuity Receipt.
10. Human, CI, repository rule, or another acceptance system decides.

28.2 Verified-Control Adapter

IntentLock, CI policy, or another controller may provide signed intent and acceptance events. ProjectMind validates and records them as evidence but does not inherit their private implementation assumptions.

28.3 Fail-Safe Handoff

Required frontier mismatch, policy mismatch, conflicted blocking claim, stale required tests, candidate/tree mismatch, or invalid receipt produces REVIEW, BLOCK, or FAIL_SAFE according to explicit policy.

Section 29 - Security Threat Model

Protected assets include evidence, repository identity, commit graph, claims, authority transitions, frontier manifests, contracts, receipts, keys, policies, and secret exclusions.

Major threats and controls:

- **Evidence poisoning:** schema validation, provenance, quarantine, contradictory evidence retention, and no text-to-authority promotion.
- **Authority laundering:** separate authenticity from authority; ledger every transition; bind transition to prior-state and frontier hashes.
- **Branch confusion:** commit reachability checks; exclude incomparable evidence.
- **Stale-context replay:** frontier, commit, policy, and expiry binding.
- **Prediction laundering:** freeze CCC before observed delta extraction.
- **Delta evasion:** independently reconstruct the candidate graph and tests.
- **Receipt forgery:** canonical encoding, hashes, optional DSSE signatures, key policy, and offline verification.
- **Prompt injection:** treat repository and tool text as untrusted; structured claims; bounded context; no instruction authority from content.
- **MCP compromise:** local stdio, sandboxing, fixed manifest, no sampling, no writes, no network, and digest-pinned binary.
- **Extractor compromise:** allowlist, sandbox, resource limits, pinned digests, SBOM, and cross-extractor conflict checks.
- **SQLite corruption:** patched SQLite version, one writer, short transactions, checkpoints, backups, quick/integrity checks, and evidence rebuild.
- **Secret leakage:** default exclusions, content minimization, redaction, output filters, and adversarial fixtures.

Residual risks include parser defects, malicious but authorized input, incomplete graphs, incorrect human decisions, weak tests, local privilege compromise, key theft, denial of service, and policy mistakes.

Section 30 - Privacy and Data Governance

ProjectMind is local-first and requires no source-code upload. It stores hashes, structural metadata, claims, evidence references, and bounded summaries; full file bodies are excluded by default.

Default exclusions include environment files, private keys, credential stores, tokens, SSH material, signing keys, browser profiles, and user-configured sensitive paths. Paths and symbol names may themselves be sensitive and remain subject to retention and export policy.

Cloud-model use is disabled by default. Enabling it requires explicit consent, provider and region disclosure, path allowlists, redaction, request budgets, retention disclosure, and request/response digests. Cloud output remains PROPOSED.

Data-subject access, deletion, retention, backup, and export behavior must be documented when personal data is processed. ProjectMind may support governance evidence but does not establish GDPR, EU AI Act, ISO, SOC 2, or other compliance.

Section 31 - Reliability, Recovery, and Concurrency

One application writer owns mutations. CLI and MCP readers use short read transactions bound to a snapshot and frontier digest.

Startup recovery:

1. verify the embedded SQLite version is patched;
2. open the database and run quick/integrity checks according to policy;
3. validate migration checksums;
4. reconcile inbox, quarantine, and processed artifacts;
5. verify the authority-ledger head;
6. verify active snapshot and frontier manifests;
7. resume or invalidate incomplete jobs;
8. emit a recovery report.

Checkpoint policy accounts for WAL size, long readers, disk space, and backup windows. Backups use the SQLite backup API or another documented safe method; the database file is never copied without its required WAL state.

`projectmind rebuild --from-evidence` must reproduce active claim, frontier, and ledger-head digests or return RECOVERED_WITH_GAPS, REBUILD_REQUIRED, REVIEW, or FAIL_SAFE.

Section 32 - Observability and Operation Evidence

Signals include ingestion and quarantine counts, branch-incomparable exclusions, extractor duration and failures, claim counts by lifecycle, conflicts, stale and incomplete claims, frontier build time, frontier reproducibility failures, MCP response size and omissions, CCC counts, unexpected deltas, receipt verification, database/WAL size, checkpoint duration, and rebuild fidelity.

Logs are structured, redacted, and correlated by repository, task, frontier, CCC, and receipt IDs. Optional OpenTelemetry traces cover ingestion, extraction, frontier computation, retrieval, contract compilation, reconciliation, and verification.

Each operation produces an Evidence Receipt containing canonical input and output hashes, snapshot before/after, status, duration, tool versions, and limitations. Operational evidence is separate from the Context and Continuity Receipts.

Section 33 - Local File Structure

```
.projectmind/
  config.yaml
  project.yaml
  graph.db
  inbox/{git,events,tests,coverage,indexes}/
  evidence/{raw,normalized,receipts}/
  commits/{manifests,reachability-cache}/
  frontiers/{manifests,active}/
  claims/{exports,conflicts}/
  authority/{ledger.jsonl,policies,approvals}/
  contracts/{draft,frozen,expired}/
  receipts/{context,continuity,operation}/
```

```
  snapshots/{database,manifests}/
  quarantine/{events,extractors,reports}/
  extractors/{registry,cache,outputs}/
  policies/{promotion,secret-exclusions,retention,mcp,reconciliation}/
  briefs/{latest,history}/
  reports/{health,validation,benchmarks,limitations}/
  exports/{graph,policy-context,sbom}/
  audit/{events,mcp-calls,human-actions}/
  runtime/{writer.lock,mcp.pid,active-snapshot.json}/
```

Repository configuration may relocate large caches, but canonical evidence, policies, contracts, and receipts remain bound by digest.

Section 34 - Command Surface

Core commands:

- `projectmind init`
- `projectmind ingest git|event|tests|coverage <artifact>`
- `projectmind index [--changed-since <commit>]`
- `projectmind graph validate|export`
- `projectmind frontier build|show|verify --commit <commit>`
- `projectmind claim explain <claim-id>`
- `projectmind authority propose|approve|reject|verify`
- `projectmind brief --task <file-or-text>`
- `projectmind context query|verify-receipt`
- `projectmind consequence compile|freeze|show`
- `projectmind reconcile --contract <id> --candidate <commit-or-tree>`
- `projectmind receipt verify <receipt>`
- `projectmind serve mcp`
- `projectmind snapshot create|verify`
- `projectmind rebuild --from-evidence`
- `projectmind backup|restore|health|report`

Later commands may add watch mode, CI integration, policy simulation, extractor installation, team synchronization, and hosted operation.

Section 35 - MCP Resources and Tool Result Contract

Approved MVP resources expose project identity, target snapshot, current frontier, enforced claims, recent failures, latest brief, graph schema, and limitations.

Approved tools include exact project identity, enforced invariants, relevant context, symbol context, dependency path, impacted tests, change history, failure memory, claim explanation, and snapshot freshness. No write or arbitrary-query tool exists.

Every tool result follows this shape:

```
{
  "target_commit": "abc123",
  "frontier_digest": "sha256:...",
  "status": "OK",
  "data": {},
  "claim_ids": [],
  "evidence_ids": [],
  "quality": {
    "confidence": "HIGH",
```

```
    "authority": "REVIEW",
    "freshness": "CURRENT",
    "completeness": "PARTIAL"
  },
  "limits": {"max_depth": 4, "max_nodes": 500},
  "omitted_source_classes": [],
  "unknowns": [],
  "context_receipt_digest": "sha256:..."
}
```

Section 36 - Canonical Exchange Schemas

36.1 Claim

```
{
  "claim_id": "CLAIM:file-imports-package:abc123",
  "subject_id": "FILE:src/app.ts",
  "predicate": "IMPORTS",
  "object": {"entity_id": "PACKAGE:auth"},
  "introduced_commit": "abc123",
  "invalidated_commit": null,
  "quality": {
    "authenticity": "HASH_BOUND",
    "confidence": "HIGH",
    "authority": "NONE",
    "freshness": "CURRENT",
    "completeness": "PARTIAL"
  },
  "lifecycle": "EVIDENCED",
  "evidence_ids": ["EVIDENCE:scip:0001"]
}
```

36.2 Authority Transition

```
{
  "transition_id": "PM-AUTH-0001",
  "claim_id": "CLAIM:INV-AUTH-002:v2",
  "prior_state_hash": "sha256:...",
  "new_state_hash": "sha256:...",
  "actor_or_policy": "human:local-owner",
  "reason": "Authentication is an approved product invariant.",
  "frontier_digest": "sha256:...",
  "occurred_at": "2026-07-13T12:30:00Z",
  "signature_ref": null
}
```

CCC, Context Receipt, and Continuity Receipt schemas are defined in Sections 26 and 57. Production schemas require JSON Schema, versioning, canonical examples, compatibility rules, and adversarial fixtures.

Section 37 - State Machines

Evidence: RECEIVED -> VERIFYING -> VALIDATED -> CANONICALIZED -> STORED, with QUARANTINED, RETRYABLE, or REJECTED branches.

Claim: PROPOSED -> EVIDENCED -> ENFORCED, with REJECTED, CONFLICTED, STALE, SUPERSEDED, INVALIDATED, and HUMAN_RESOLVED transitions recorded separately.

Frontier: BUILDING -> VALIDATING -> READY -> ACTIVE -> HISTORICAL, with INVALID or REBUILD_REQUIRED failure states.

CCC: DRAFT -> VALIDATING -> FROZEN -> EVALUATING -> SATISFIED | REQUIRES_REVIEW | VIOLATED | EXPIRED. A FROZEN contract is immutable.

Receipt: ASSEMBLING -> CANONICALIZED -> HASHED -> SIGNED_OPTIONAL -> VERIFIED | INVALID | INCOMPLETE.

MCP: STOPPED -> STARTING -> READY -> SERVING -> DRAINING -> STOPPED, with DEGRADED and LOCKDOWN states.

Section 38 - Failure Modes and Required Behavior

- Missing verified event: continue Git-native with `intent=UNKNOWN`.
- Invalid signature or schema: quarantine; no active claims.
- Missing or malformed commit: incomplete frontier; no silent substitution.
- Branch-incomparable evidence: exclude from active view and disclose.
- SCIP unavailable: use declared fallback and mark semantic completeness partial.
- Parser failure: preserve other evidence and mark affected scope UNKNOWN.
- Aggregate coverage only: create suite-level execution claims only.
- Ambiguous test or ADR text: PROPOSED or no claim; never auto-enforce.
- Authority-ledger mismatch: disable blocking export and require review.
- Frontier mismatch during task: expire CCC or require explicit rebase.
- CCC mutation after freeze: invalidate evaluation.
- Candidate/tree mismatch: invalidate Continuity Receipt.
- SQLite writer contention: bounded retry/queue, then visible overload.
- WAL growth or unpatched version: health failure and release block.
- Secret detected: stop affected export, redact, record event, and review.
- Graph or rebuild inconsistency: disable blocking export and fail safe.

Section 39 - Proof-Test Suite 1-60

ProjectMind may not be called implementation-ready until these tests pass.

Event and Provenance

1. Valid signed merge event is ingested.
2. Invalid signature is quarantined.
3. Duplicate event is idempotent.
4. Event ID collision with different hash is blocked.
5. Repository mismatch is blocked.
6. Missing commit creates incomplete status.
7. Reverted merge closes affected temporal facts.
8. CloudEvents extension fields survive normalization.

Graph Store

9. Foreign-key integrity passes.
10. Recursive dependency query returns bounded path.

11. Historical query reconstructs commit state.
12. Snapshot hash changes after valid update.
13. Migration rollback restores prior schema.
14. Rebuild from evidence reproduces graph counts and hashes.
15. WAL checkpoint policy prevents unbounded growth.
16. Concurrent readers do not see partial writes.

Extractors

17. SCIP index creates definitions and references.
18. Invalid SCIP path is rejected.
19. Tree-sitter fallback extracts imports.
20. Malformed file does not crash scan.
21. Build metadata creates package and target nodes.
22. Changed-file indexing invalidates old facts.
23. External extractor timeout is contained.
24. Extractor with wrong digest is rejected.

Tests and Invariants

25. JUnit result creates test nodes.
26. Aggregate LCOV creates suite-level coverage only.
27. Per-test coverage creates test-case edges.
28. Test name creates PROPOSED candidate.
29. Passing assertion creates EVIDENCED candidate.
30. No LLM-created invariant becomes ENFORCED.
31. Human approval promotes invariant.
32. Deleted supporting test marks invariant stale.
33. Skipped required test reduces health.
34. Test deletion in a diff triggers review.

MCP

35. MCP server exposes only approved tools.
36. MCP path traversal is rejected.
37. Arbitrary SQL is impossible.
38. Output-size limit truncates safely.
39. Tool-manifest hash is stable.
40. MCP response contains provenance and snapshot.
41. Server refuses unbound repository root.
42. Read-only server cannot modify repository.
43. No sampling capability is advertised in MVP.
44. Cancellation terminates expensive query.

Consequence and Integration

45. High-centrality module change raises risk.
46. Enforced invariant exports as blocking policy.
47. Proposed invariant does not export as blocking.
48. Snapshot mismatch causes independent acceptance handoff failure.
49. Roadmap conflict produces review decision.
50. Prior repeated failure appears in the next brief.
51. Out-of-band Git change is disclosed.
52. Pre-merge and post-merge graph deltas agree.

Security and Privacy

53. `.env` contents never enter graph output.
54. Private-key fixture is redacted.
55. Malicious README instruction remains untrusted data.
56. Poisoned MCP arguments cannot escape root.
57. Oversized event is rejected.
58. Symlink escape is rejected.
59. Raw LLM output cannot alter authority fields.
60. Quarantine artifacts remain separate from active evidence.

Section 40 - Benchmark and Evaluation Plan

Evaluate four conditions on published repositories and tasks:

1. file search without persistent memory;
2. static project instructions or brief;
3. persistent structural graph or event memory;
4. full ProjectMind frontier, authority, contract, reconciliation, and receipts.

Measurements:

- relevant-context precision and recall;
- impacted-entity and impacted-test precision/recall;
- regression rate and unrelated-file rate;
- false warning, false review, and false block rates;
- stale or branch-incomparable evidence detection;
- unexpected graph-delta detection;
- Evidence Frontier reproducibility;
- Authority Promotion Ledger completeness;
- Context and Continuity Receipt verification;
- rebuild fidelity;
- initial/incremental index time, memory, database size, and query p50/p95/p99;
- context tokens, tool calls, human corrections, and rollback frequency.

Report repository commit IDs, hardware, software versions, extractors, policies, commands, raw outputs, confidence intervals where appropriate, and every excluded task. No performance or safety claim is allowed without released evidence.

Section 41 - MVP Boundary

ProjectMind CLI v0.1 is Linux-first, local-first, Git-native, and single-user.

MVP scope:

- Rust command-line application;
- patched SQLite with STRICT tables;
- TypeScript/JavaScript repositories;
- Git, package.json, tsconfig, SCIP when available, and Tree-sitter fallback;
- JUnit XML, LCOV/Istanbul, and optional per-test contexts;
- Evidence Frontier and authority-transition ledger;
- read-only local MCP;
- Context Receipt, CCC, graph-delta reconciliation, and Continuity Receipt;
- JSON, Markdown, and optional DSSE exports.

The MVP excludes autonomous code modification, remote MCP, hosted services, multi-user authorization, all-language support, complete semantic analysis, automatic LLM enforcement, and production certification.

Section 42 - MVP Acceptance Criteria

The MVP is acceptable only when all required evidence exists for:

1. idempotent repository initialization;
2. reversible schema migrations;
3. patched SQLite version verification;
4. atomic evidence ingestion and quarantine;
5. commit-parent graph construction;
6. branch reachability and incomparable-branch tests;
7. claim/evidence many-to-many integrity;
8. deterministic Evidence Frontier generation;
9. stable RFC 8785 frontier digest;
10. separate quality-vector fields;
11. append-only authority-transition verification;
12. no model-generated self-promotion;
13. SCIP ingestion and Tree-sitter fallback;
14. build and dependency graph generation;
15. aggregate versus per-test coverage distinction;
16. negative test evidence and staleness;
17. bounded impact query;
18. Context Receipt generation and verification;
19. CCC freeze before observed analysis;
20. predicted/observed delta reconciliation;
21. unexpected high-authority impact detection;
22. Continuity Receipt generation and offline verification;
23. stale-frontier replay rejection;
24. monotonic-caution property tests;
25. read-only MCP and path/symlink containment;
26. manifest mutation detection;
27. secret-exclusion adversarial tests;
28. database backup, crash recovery, and evidence rebuild;
29. fuzzing for event, parser, path, and receipt inputs;
30. released benchmark and limitations reports;
31. SBOM and signed release evidence;
32. independent security review before any production-ready claim.

Section 43 - Implementation Roadmap

The roadmap is evidence-gated, not date-promised.

- **Phase 0 - Research and claim lock:** fixtures, threat model, prior-art matrix, schema decisions, and benchmark protocol.
- **Phase 1 - Evidence and commit DAG:** repository identity, commits, parents, canonical evidence, quarantine, and migrations.
- **Phase 2 - Claims and frontier:** entities, claims, evidence links, reachability, conflicts, completeness, frontier manifest, and digest.
- **Phase 3 - Extractor fabric:** Git/build adapters, SCIP, Tree-sitter, receipts, and incremental invalidation.
- **Phase 4 - Test evidence:** test results, aggregate/per-test execution, staleness, skipped/deleted tests, and impacted tests.
- **Phase 5 - Authority governance:** policies, human actions, promotion ledger, conflicts, monotonic-caution tests, and export.
- **Phase 6 - Context:** briefs, adaptive queries, local MCP, limits, omissions, and Context Receipts.
- **Phase 7 - Consequence and reconciliation:** CCC, predicted graph delta, observed reindex, mismatch rules, and Continuity Receipt.

- **Phase 8 - Hardening:** fuzzing, performance, backup/rebuild, SBOM, signed artifacts, limitations, and external review.

A ten-day feasibility spike may prove the vertical slice. A credible v0.1 should be scheduled only after the spike measures repository scale, extractor quality, and implementation capacity.

Section 44 - Technical Stack and Version Policy

Selected core: Rust, Git, SQLite, JSON/JSON Schema, and local stdio MCP.

Candidate libraries include clap, serde, a JSON Schema validator, rusqlite, cryptographic hashes, optional Ed25519/DSSE support, tracing, Tokio where justified, SCIP bindings, and Tree-sitter bindings. Every dependency requires current license, provenance, maintenance, advisory, install, rollback, and SBOM review.

SQLite must resolve to 3.51.3+ or an official corrected backport. The build records the actual SQLite runtime version and compile options. Extractors such as scip-typescript run as digest-pinned, resource-limited, network-denied adapters.

Canonical JSON uses RFC 8785. Provenance exports may map to W3C PROV and in-toto statements. Policy may be implemented with a small deterministic core or a reviewed OPA/Cedar adapter; ProjectMind semantics must not depend on ambiguous model output.

Section 45 - Open-Source Dependency Decisions

45.1 Five Architecture Branches Considered

Branch A - TypeScript Core + ts-morph

Advantages:

- fastest TS/JS MVP;
- easy AST access;
- simple MCP SDK path.

Weaknesses:

- language lock-in;
- Node runtime;
- duplicated semantic-index work.

Decision: not selected as the hardened core.

Branch B - TypeScript Core + SCIP

Advantages:

- standardized semantic index;
- strong TS/JS support.

Weaknesses:

- external indexer dependence;
- not automatically polyglot;
- less suitable for a single hardened binary.

Decision: retained as adapter path.

Branch C - Rust Core + Tree-sitter

Advantages:

- single binary;

- polyglot syntax;
- fast incremental parsing;
- strong local-tool profile.

Weaknesses:

- syntax is not compiler semantics;
- grammar packaging and maintenance.

Decision: selected as fallback extraction foundation.

Branch D - Joern Code Property Graph

Advantages:

- rich syntax, control-flow, and data-flow representation;
- security analysis capability.

Weaknesses:

- heavier runtime;
- operational complexity;
- broad dependency footprint;
- unsuitable for minimal MVP.

Decision: advanced optional adapter.

Branch E - Hybrid Rust Core + SQLite + SCIP + Tree-sitter + Build Evidence

Advantages:

- semantic accuracy where available;
- resilient fallback;
- local-first;
- extensible;
- evidence diversity;
- no single extractor monopoly.

Weaknesses:

- more adapter work;
- conflict handling required.

Decision: selected.

45.2 OSS Admission Gate

A dependency may enter the core path only after license identification, provenance, maintenance review, advisory review, install and rollback tests, test presence, stack compatibility, and SBOM entry creation.

Unknown license or provenance blocks core-path adoption.

Section 46 - Open-Source Governance

46.1 Recommended License

Apache-2.0.

Reason:

- permissive use;

- explicit patent grant;
- suitable for infrastructure adoption.

This is not legal advice. Human license review is required.

46.2 Required Repository Files

- `README.md`
- `LICENSE`
- `NOTICE`
- `SECURITY.md`
- `CONTRIBUTING.md`
- `CODE_OF_CONDUCT.md`
- `GOVERNANCE.md`
- `ROADMAP.md`
- `CHANGELOG.md`
- `THREAT_MODEL.md`
- `docs/limitations.md`
- `docs/architecture.md`
- `docs/event-schema.md`
- `docs/mcp-security.md`

46.3 Security Baseline

The project should align its hygiene with NIST SSDF concepts, OpenSSF OSPS Baseline, OpenSSF Scorecard, signed releases, dependency review, branch protection, responsible disclosure, and SBOM generation.

No certification claim should be made without review.

46.4 Contribution Model

Contributions must preserve the PROPOSED vs ENFORCED boundary, include tests, update schemas and migrations, document extractor limitations, avoid network egress by default, include license provenance, and update the threat model when attack surface changes.

Section 47 - Future Team and Hosted Architecture

The local MVP remains canonical.

Future team mode may add:

- central policy registry;
- shared approvals;
- signed snapshot exchange;
- repository fleet view;
- project ownership;
- reviewer roles;
- evidence retention;
- self-hosted server;
- encrypted synchronization.

Cloud sync must not become mandatory.

Hosted mode requires tenant isolation, authentication, authorization, encryption, audit, retention controls, incident response, regional deployment decisions, and legal review.

Section 48 - Public Positioning and Claim Rules

Allowed claims:

- proposed evidence-frontier architecture for coding-agent continuity;
- commit-DAG-bound project context;
- separate evidence quality and enforcement authority;
- append-only authority transitions;
- immutable consequence contracts;
- predicted/observed graph-delta reconciliation;
- verifiable context and continuity receipts;
- local-first and Git-native MVP design.

Claims requiring implementation evidence:

- reduces regressions or context use;
- improves agent accuracy;
- detects a stated percentage of unexpected impacts;
- scales to a repository class;
- is secure, reliable, or production-ready.

Prohibited claims without formal support:

- first, only, patented, patent-pending, impossible to bypass, complete project understanding, guaranteed correctness, legally compliant, certified, or peer reviewed.

Section 49 - Ten-Day Feasibility Spike

Day 1: repository identity, commit DAG, and fixtures. Day 2: evidence schema, canonicalization, and quarantine. Day 3: claims, evidence links, and active graph view. Day 4: Evidence Frontier and branch-incomparability tests. Day 5: SCIP/Tree-sitter/build extraction on one TypeScript repository. Day 6: JUnit/coverage ingestion and impacted-test query. Day 7: authority transition ledger and monotonic-caution properties. Day 8: Context Receipt and three read-only MCP tools. Day 9: CCC, observed delta, and reconciliation. Day 10: Continuity Receipt, rebuild test, benchmark capture, and honest report.

The spike succeeds only if it produces executable evidence for one end-to-end fixture. It does not establish product readiness.

Section 50 - Final Product Statement

ProjectMind is a proposed local-first continuity-control architecture for AI coding agents. It computes a branch-correct Evidence Frontier, governs the promotion of project claims, records the exact context supplied to an agent, freezes expected consequences before evaluation, compares those expectations with the observed repository and test delta, and produces a verifiable Continuity Receipt for an independent acceptance decision.

It is not a new parser, code graph, MCP protocol, test selector, policy engine, or attestation format. Its proposed contribution is the verifiable chain connecting these existing capabilities to repository-state continuity.

Section 51 - Final MVP Statement

The first implementation target is a Linux-first Rust CLI using Git, patched SQLite, TypeScript/JavaScript extraction, test evidence, and a read-only local MCP interface.

The vertical slice must prove:

```
base commit
  -> Evidence Frontier
  -> Context Receipt
  -> Change Consequence Contract
  -> frozen candidate
  -> observed graph/test delta
  -> reconciliation
  -> Continuity Receipt
  -> independent decision
```

No COMPLETE status is allowed without executed tests and retained receipts.

Section 52 - Implementation Agent Execution Specification

Role

Build ProjectMind CLI v0.1 as the observer, project-state, context, consequence, and receipt layer. Do not build a coding agent or silently add merge authority.

Mandatory Order

1. Pre-audit the repository, toolchain, tests, CI, licenses, and secret-bearing paths by name only.
2. Implement evidence and commit-DAG storage before graph retrieval.
3. Implement claims and many-to-many evidence links before authority.
4. Implement deterministic Evidence Frontiers before MCP.
5. Implement authority transitions and property tests before blocking export.
6. Implement Context Receipts before consequence contracts.
7. Freeze CCCs before observed-delta analysis.
8. Implement offline Continuity Receipt verification before release packaging.

Required Modules

- `repository`: identity, commits, parents, reachability;
- `evidence`: canonicalization, digest, ingestion, quarantine;
- `claims`: entities, claims, evidence links, active views;
- `frontier`: admissibility, conflicts, completeness, canonical manifest;
- `authority`: policy evaluation and append-only transitions;
- `extractors`: Git/build, SCIP, Tree-sitter, tests, coverage;
- `context`: queries, limits, omissions, briefs, MCP, Context Receipts;
- `consequence`: CCC construction and freeze;
- `reconcile`: observed extraction and predicted/observed comparison;
- `receipts`: canonical encoding, hash, optional DSSE signing, offline verify;
- `operations`: migrations, backup, rebuild, health, SBOM, release evidence.

Release Blocks

Release is blocked by unpatched SQLite, non-reproducible frontier hashes, model self-promotion, branch-incomparable evidence leakage, missing omissions in a Context Receipt, post-observation CCC mutation, invalid receipt verification, secret leakage, failed rebuild fidelity, or unsupported public claims.

Validation Ladder

Formatting -> compile -> unit tests -> schema/migration tests -> property tests -> integration tests -> adversarial/fuzz tests -> end-to-end fixture -> benchmark -> SBOM and release

evidence.

Final Report

```
STATUS: COMPLETE | PARTIAL | BLOCKED | FAILED
BASE COMMIT:
FILES CHANGED:
COMMANDS RUN:
TESTS PASSED:
TESTS FAILED:
EVIDENCE CREATED:
FRONTIER DIGEST:
CCC DIGEST:
CONTINUITY RECEIPT:
SECURITY FINDINGS:
LIMITATIONS:
ROLLBACK:
NEXT TASK:
```

Section 53 - Claimed Contribution and Novelty Boundary

The public contribution is a proposed architecture, not a patent claim.

The strongest candidate contribution is the following linked method:

> Compute a canonical Evidence Frontier for a target repository commit; derive an authority-governed, task-specific consequence contract from that frontier; record the exact context delivered; independently reconstruct the candidate project-state delta; reconcile predicted and observed effects; and bind the complete chain into a verifiable continuity receipt.

No single component is claimed as new. The research question is whether this specific chain produces a useful technical effect: reproducible, branch-correct, independently reviewable project context and consequence evidence.

Patentability is unknown. A formal claim chart and professional patent search would need to compare each essential limitation against patents, applications, papers, products, and public code published before any filing date.

Section 54 - Evidence Frontier Algorithm

Inputs: repository ID, target commit `C`, policy version `P`, extractor registry `X`, and retained evidence set `E`.

```
1. Verify repository identity and that C exists.
2. Construct or verify the parent DAG reachable from C.
3. Validate and canonicalize candidate evidence objects.
4. Keep evidence whose repository scope matches C.
5. Keep commit-bound evidence only when its binding commit is reachable from C.
6. Exclude branch-incomparable evidence from the active view.
7. Apply reachable invalidations and supersession records.
8. Retain contradictions and mark affected claims CONFLICTED.
9. Evaluate freshness and required evidence classes under P.
10. Materialize active claims without collapsing quality-vector dimensions.
11. Sort the manifest by stable identifiers.
12. Serialize with RFC 8785 JCS and compute frontier_digest.
```

Determinism requirement: identical repository objects, retained evidence, policy, extractor versions, and schema must produce identical manifest bytes and digest.

Section 55 - Authority Promotion Ledger

Every authority transition is a hash-linked record containing the claim ID, prior state hash, new state hash, Evidence Frontier digest, actor or policy identity, reason, effective scope, time, and optional signature.

Verification checks:

- transition chain is complete and ordered;
- prior-state hash matches the recorded claim state;
- actor or policy possessed the declared authority;
- supporting and contradicting evidence were visible;
- scope and expiry are honored;
- no generated content promoted itself;
- ledger head matches the value bound into the CCC and Continuity Receipt.

The ledger establishes traceability of authority decisions, not their wisdom.

Section 56 - Predicted-versus-Observed Delta Reconciliation

The CCC contains a predicted consequence envelope. After the candidate is frozen, ProjectMind independently extracts an observed delta from the candidate tree, builds, tests, coverage, and policy evidence.

Reconciliation classes:

- MATCHED_EXPECTATION;
- ALLOWED_UNPREDICTED;
- REVIEW_UNPREDICTED;
- FORBIDDEN_EFFECT;
- REQUIRED_EFFECT_MISSING;
- TEST_EVIDENCE_INSUFFICIENT;
- FRONTIER_OR_POLICY_MISMATCH;
- UNKNOWN.

The comparison is set- and rule-based. An optional score may summarize results but cannot replace the enumerated mismatches. A changed public interface, protected dependency, required test, authority-bearing claim, or high-centrality relation outside the predicted envelope triggers at least REVIEW.

Section 57 - Context and Continuity Receipts

A Context Receipt binds the agent-visible state: target commit, frontier, query, returned claims, evidence, limits, omissions, tool manifest, and response digest.

A Continuity Receipt binds the evaluation chain:

```
{
  "schema": "dev.projectmind.continuity-receipt/v1",
  "repository": "sha256:...",
  "base_commit": "abc123",
  "candidate_tree_or_commit": "def456",
  "evidence_frontier_digest": "sha256:...",
  "authority_ledger_head": "sha256:...",
```

```
  "context_receipt_digests": ["sha256:..."],
  "consequence_contract_digest": "sha256:...",
  "observed_delta_digest": "sha256:...",
  "test_evidence_digests": ["sha256:..."],
  "reconciliation": "REQUIRES_REVIEW",
  "mismatches": ["unexpected public API removal"],
  "limitations": ["runtime topology not observed"],
  "receipt_digest": "sha256:..."
}
```

Receipts are canonical JSON and may be wrapped in DSSE. Offline verification checks bytes, bindings, ledger head, key policy, and artifact availability.

Section 58 - Falsifiability and Proof Tests 61-80

61. Evidence from an incomparable branch is excluded from the active frontier.
62. Merged evidence becomes eligible only through reachable commit ancestry.
63. Identical inputs reproduce identical frontier bytes.
64. A policy-version change changes the frontier or records semantic equivalence.
65. Contradictory evidence remains visible and produces CONFLICTED.
66. Signature verification cannot raise policy authority by itself.
67. Every ENFORCED transition has a valid prior-state hash.
68. A generated summary cannot be its own promotion basis.
69. Evidence loss cannot reduce REVIEW/BLOCK to PASS.
70. A Context Receipt lists all applied truncation limits.
71. A tool-description change mutates the manifest hash.
72. A CCC cannot be modified after observed analysis begins.
73. Required predicted effects missing from the candidate are reported.
74. Unexpected high-authority effects trigger review.
75. Aggregate coverage never becomes per-test evidence.
76. Candidate/tree mismatch invalidates the Continuity Receipt.
77. Ledger-head mismatch invalidates the Continuity Receipt.
78. Offline verification succeeds without access to an LLM.
79. Rebuild from evidence reproduces active claim and frontier digests.
80. Unpatched SQLite fails the release-version gate.

Section 59 - Publication, Intellectual Property, and Research Status

Status: proposed independent technical architecture; implementation not supplied; proof tests not executed; benchmarks not run; security review not performed; not peer reviewed; patentability and freedom to operate unknown.

Public disclosure may become prior art and may restrict later patent options in many jurisdictions. The author should decide whether to seek professional patent advice before public release. This document is not legal advice.

Section 60 - References

1. Vogel, M., Meyer-Eschenbach, F., Kohler, S., Gruenewald, E., and Balzer, F. (2026). Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via

MCP. https://arxiv.org/abs/2603.27277
2. Cherny-Shahar, T., and Yehudai, A. (2026). Repository Intelligence Graph: Deterministic Architectural Map for LLM Code Assistants. https://arxiv.org/abs/2601.10112
3. Malo, R. C., and Qiu, T. (2026). PROJECTMEM: A Local-First, Event-Sourced Memory and Judgment Layer for AI Coding Agents. https://arxiv.org/abs/2606.12329
4. Yang, B., et al. (2025). Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. https://arxiv.org/abs/2503.21710
5. Chen, Z., et al. (2025). Prometheus: Unified Knowledge Graphs for Issue Resolution in Multilingual Codebases. https://arxiv.org/abs/2507.19942
6. Xu, D., et al. (2026). RepoDoc: A Knowledge Graph-Based Framework to Automatic Documentation Generation and Incremental Updates. https://arxiv.org/abs/2604.26523
7. Rasmussen, P., et al. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. https://arxiv.org/abs/2501.13956
8. Bairi, R., et al. (2023). CodePlan: Repository-Level Coding Using LLMs and Planning. https://arxiv.org/abs/2309.12499
9. Vasilopoulos, A. (2026). Codified Context: Infrastructure for AI Agents in a Complex Codebase. https://arxiv.org/abs/2602.20478
10. Gu, S., and Mesbah, A. (2025). Fine-Grained Assertion-Based Test Selection. https://arxiv.org/abs/2403.16001
11. W3C. (2013). PROV-O: The PROV Ontology. https://www.w3.org/TR/prov-o/
12. CloudEvents Authors. (2026). CloudEvents Specification. https://github.com/cloudevents/spec/blob/main/cloudevents/spec.md
13. Secure Systems Lab. (2026). Dead Simple Signing Envelope Protocol. https://github.com/secure-systems-lab/dsse/blob/master/protocol.md
14. in-toto Authors. (2026). in-toto: Securing the Integrity of Software Supply Chains. https://in-toto.io/
15. SLSA Authors. (2026). SLSA v1.2 Provenance. https://slsa.dev/spec/v1.2/provenance
16. Rundgren, A., Jordan, B., and Erdtman, S. (2020). RFC 8785: JSON Canonicalization Scheme. https://www.rfc-editor.org/rfc/rfc8785.html
17. Model Context Protocol Authors. (2025). Model Context Protocol Specification 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18
18. Model Context Protocol Authors. (2026). Security Best Practices. https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices
19. SCIP Authors. (2026). SCIP Code Intelligence Protocol. https://github.com/scip-code/scip
20. Tree-sitter Authors. (2026). Tree-sitter Documentation. https://tree-sitter.github.io/tree-sitter/
21. GitHub. (2026). CodeQL Data Flow Analysis. https://codeql.github.com/docs/writing-codeql-queries/about-data-flow-analysis/
22. Joern Authors. (2026). Code Property Graph Documentation. https://docs.joern.io/code-property-graph/
23. SQLite Authors. (2026). Write-Ahead Logging. https://sqlite.org/wal.html
24. SQLite Authors. (2026). STRICT Tables. https://sqlite.org/stricttables.html
25. Git Authors. (2026). gitrevisions Documentation. https://git-scm.com/docs/gitrevisions
26. Open Policy Agent Authors. (2026). Rego Policy Language. https://www.openpolicyagent.org/docs/policy-language
27. Cedar Authors. (2026). Cedar Policy Language Reference Guide. https://docs.cedarpolicy.com/
28. National Institute of Standards and Technology. (2022). SP 800-218: Secure Software Development Framework 1.1. https://csrc.nist.gov/pubs/sp/800/218/final
29. OpenSSF. (2026). Open Source Project Security Baseline. https://baseline.openssf.org/
30. OWASP Foundation. (2026). CycloneDX Specification Overview. https://cyclonedx.org/specification/overview/
31. Coverage.py Authors. (2026). Measurement Contexts. https://coverage.readthedocs.io/en/latest/contexts.html
32. Istanbul Authors. (2026). Istanbul JavaScript Coverage. https://istanbul.js.org/
33. OWASP GenAI Security Project. (2025). LLM01: Prompt Injection. https://genai.owasp.org/llmrisk/llm01-prompt-injection/

34. World Intellectual Property Organization. (2026). Frequently Asked Questions: Patents. https://www.wipo.int/en/web/patents/faq_patents

Publication Completion Record

- **Edition:** v4.0 Public Research Edition
- **Document status:** Complete publication candidate
- **Research status:** Proposed architecture; implementation and empirical validation remain future work
- **Author responsibility:** Agim Haxhijaha retains responsibility for the claims, omissions, and release decision
- **Release control:** Public release remains subject to the intellectual-property decision gate described in Section 59

END OF PUBLICATION.